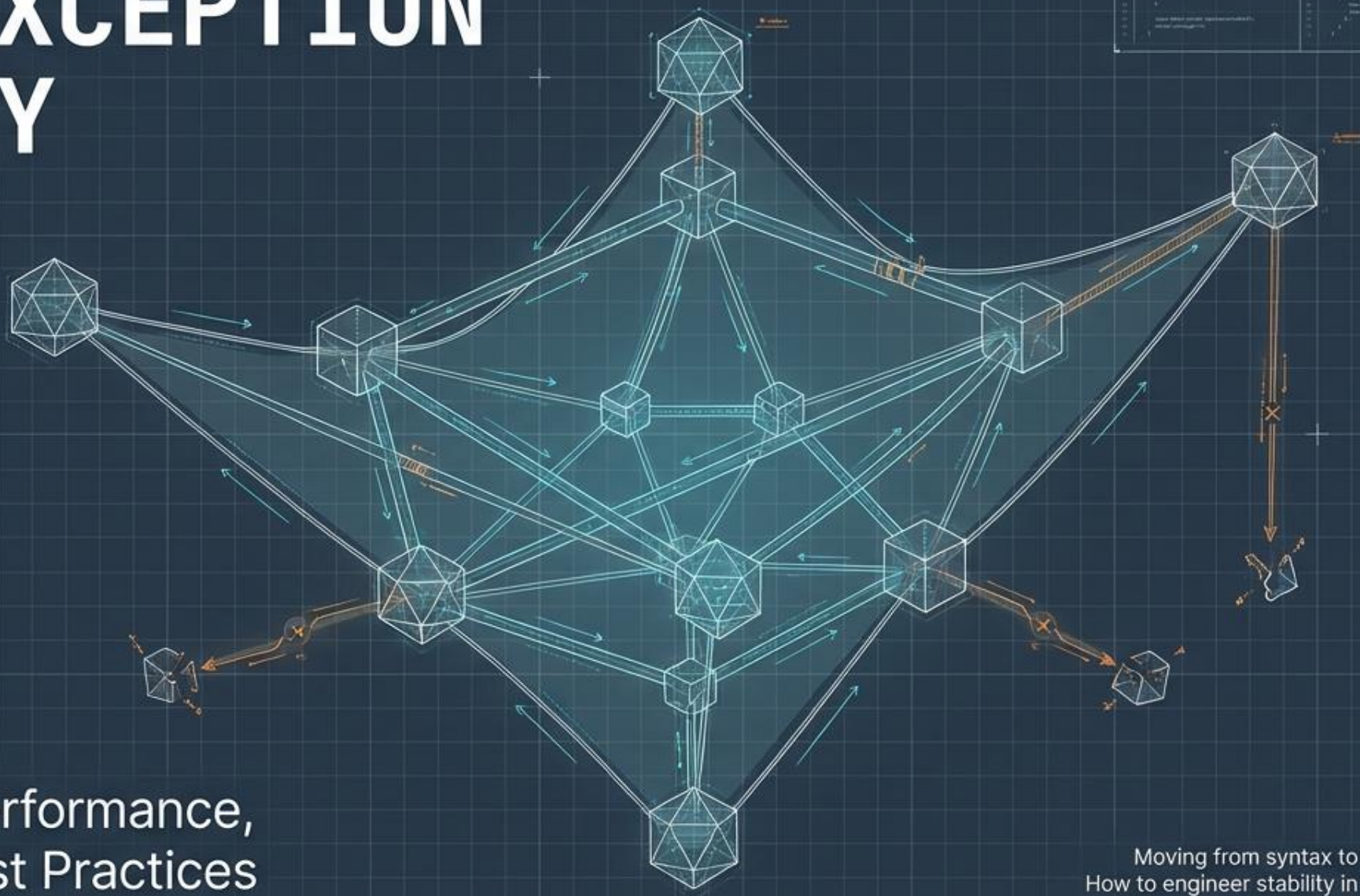


JAVA EXCEPTION MASTERY

STATUS: STABLE
JVM VERSION: 17+
EXCEPTION COUNT: MINIMAL
THROUGHPUT: OPTIMIZED



Architecture, Performance,
and Modern Best Practices

Moving from syntax to strategy:
How to engineer stability in the JVM.



Advanced Programming Applications

Lecture 03- Exeption Handling

Dr. Mohamed Mostafa Eltaweel



What are Exceptions?



- **Exception (Java):** An event that occurs during program execution which disrupts the normal flow of instructions.
- **Java Exception Handling:** A robust, object-oriented mechanism in Java used to detect, manage, and respond to runtime errors gracefully.
- **Common Causes of Exceptions:**
 - Invalid user input
 - Hardware failures
 - Network connectivity issues
 - Database server downtime
- **Exception Handling Code:** Specialized code that defines how the program should respond to specific error conditions to prevent crashes and maintain normal execution flow.

Why It Is Important

- When an error occurs, the normal execution flow stops.
- Without exception handling, the program terminates immediately.
- With exception handling, we can define an **alternative path** so the program continues running or exits gracefully:
 - **Graceful termination means:**
 - The program does not crash unexpectedly., and resources (files, database connections, network connections) are properly closed.
 - A meaningful error message is shown to the user, while the remaining parts of the program can continue if possible.
- Exception handling provides a **structured mechanism to detect, manage, and recover from runtime errors**, ensuring smoother execution and better user experience.

Examples of Causes of Exceptions

- **Invalid User Input**

- **Example:**

A user enters "abc" instead of a number when the program expects an integer.

```
int age = Integer.parseInt("abc"); // Causes NumberFormatException
```

- **Hardware Failures**

- **Example:**

Trying to read a file from a USB drive that was suddenly removed.

```
FileReader file = new FileReader("E:/data.txt"); // May cause FileNotFoundException
```

Examples of Causes of Exceptions

- **Network Connectivity Issues**

- **Example:**

Trying to connect to a website without internet access.

```
URL url = new URL("https://example.com");  
URLConnection conn = url.openConnection();  
conn.getInputStream(); // May cause IOException
```

- **Database Server Downtime**

- **Example:**

Trying to connect to a database that is not running.

```
Connection conn = DriverManager.getConnection(  
    "jdbc:mysql://localhost:3306/testdb", "root", "password"); // SQLException occurs
```

Throwing and Catching Exceptions

- When an error occurs during execution, **Java creates an exception object**.
- The exception object contains debugging details such as:
 - ✓ Exception type
 - ✓ Error message
 - ✓ Line number
 - ✓ Method call hierarchy (stack trace)
- Creating the exception object and passing it to the runtime system is called **throwing the exception**.

Exception Handling Process

- When an exception is thrown, the **normal program flow stops**.
- The **Java Runtime Environment (JRE)** searches for a matching **exception handler** (catch block).
- The search starts in the **current method** where the exception occurred.
- If no handler is found, the search moves up the **call stack** (caller methods).

Example: If call stack is $A \rightarrow B \rightarrow C$ and error occurs in C, search moves:

$C \rightarrow B \rightarrow A$

Throwing and Catching Exceptions

```
public class DebugExample {  
    public static void main(String[] args)  
    {  
        methodA();  
    }  
  
    static void methodA() {  
        methodB();  
    }  
  
    static void methodB() {  
        int result = 10 / 0;    // Error  
occurs here  
    }  
}
```

```
Exception in thread "main"  
java.lang.ArithmeticException: / by zero  
    at  
    DebugExample.methodB(DebugExample.java:13)  
    at  
    DebugExample.methodA(DebugExample.java:9)  
    at DebugExample.main(DebugExample.java:5)
```

Exception Type:

java.lang.ArithmeticException

Error Message:

/ by zero

Line Number:

DebugExample.java:13

(The exact line where the error occurred)

Method Call Hierarchy (Stack Trace):

methodB → methodA → main

Throwing and Catching Exceptions

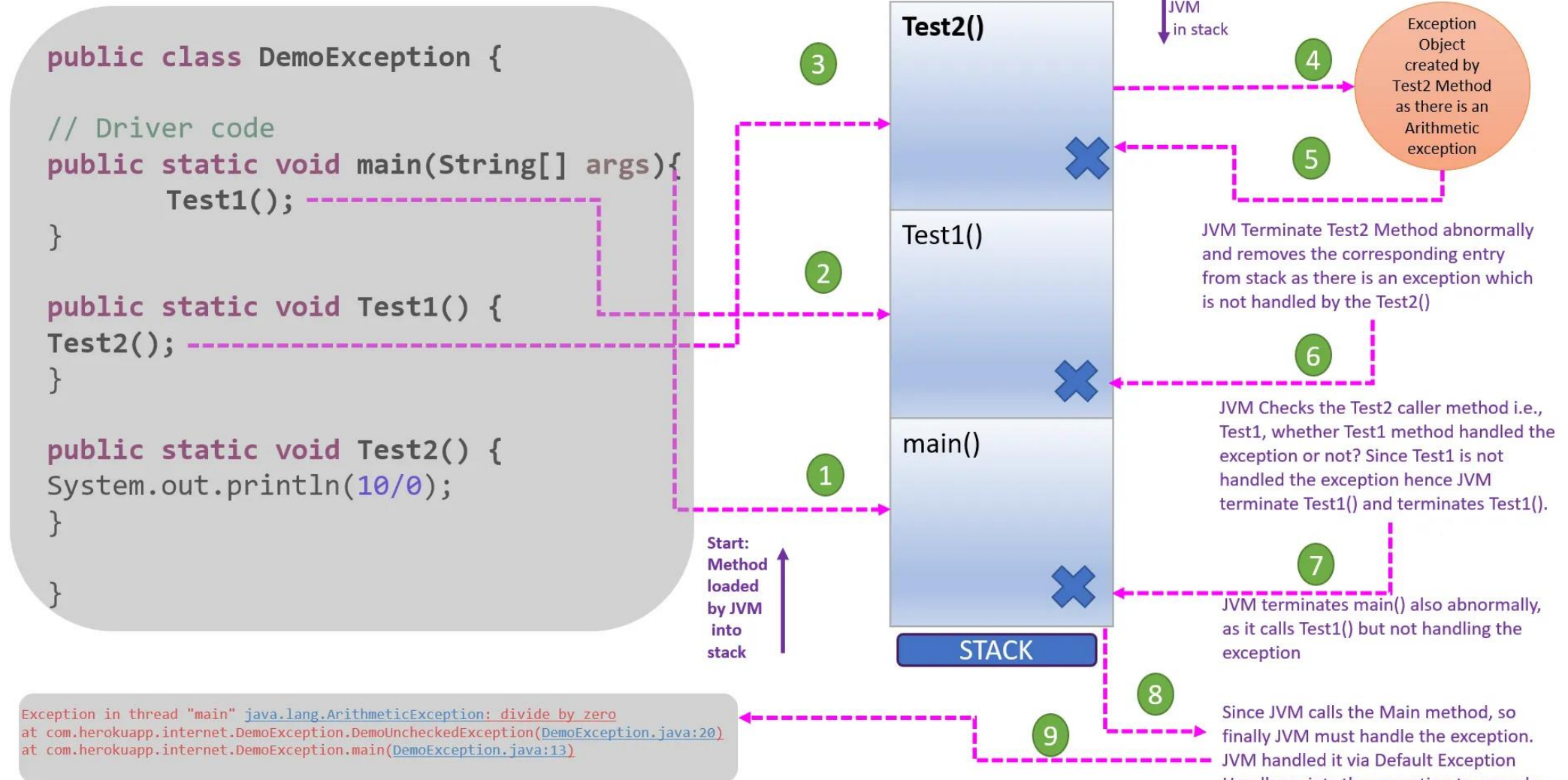


Fig: Internal Working Of How JVM Handles Exception in Java~ by Deepti Swain

Exceptions Types

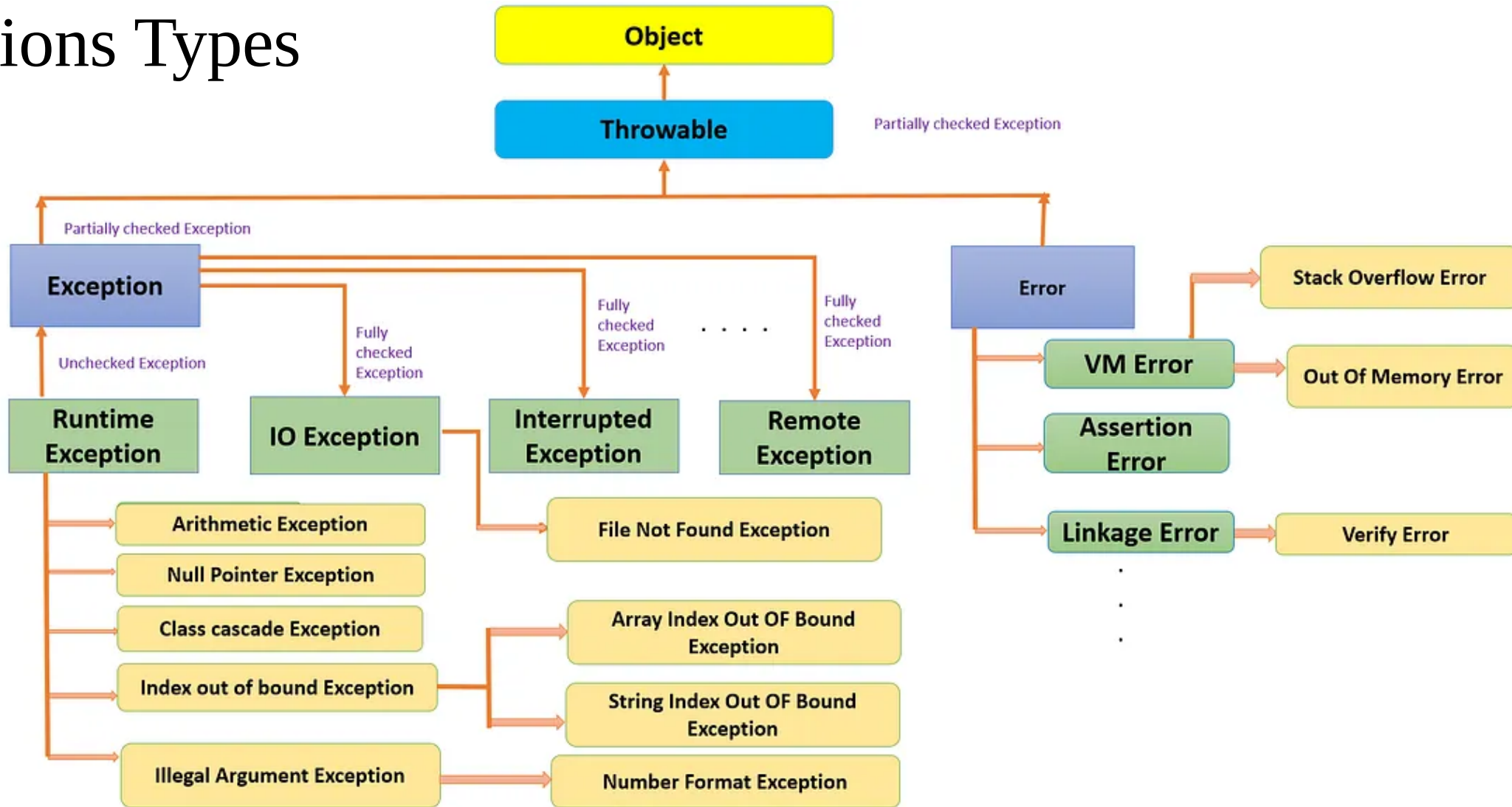


Fig. Exception Hierarchy in Java ~ by Deepthi Swain

Exceptions Types

CHECKED VS. UNCHECKED: THE GREAT DEBATE

CHECKED EXCEPTIONS



FILE / NETWORK SERVER

Predictable but unpreventable external failures.

Constraint: Compiler forces handling via try-catch or throws.

Pro: Robust APIs declaring failure modes.

Con: Boilerplate code; clutters signatures.

UNCHECKED (RUNTIME)



BUG / NULL SYMBOL (Ø)

Logic errors and programming bugs.

Constraint: No compiler enforcement.

Insight: Preferred by modern frameworks (Spring) to reduce noise.

Examples: NullPointerException, IndexOutOfBoundsException.

Oracle Guidance: If a client can reasonably recover, make it Checked. If they cannot, make it Unchecked.

Exceptions Types

The Two Types of Exception in Java:

Feature	Checked Exceptions	Unchecked Exceptions
Compiler Check	Checked at compile time	Not checked by compiler
Handling Requirement	Must be handled (try-catch) or declared using throws	Handling is optional
When They Occur	Occur during runtime but verified at compile time	Occur at runtime
Class Hierarchy	All exceptions except RuntimeException and Error classes	RuntimeException and its subclasses, Error and its subclasses
Purpose	Represent recoverable conditions (e.g., file, network issues)	Represent programming errors or serious problems
Examples	FileNotFoundException, InterruptedException, IOException	NullPointerException, ArithmeticException, ArrayIndexOutOfBoundsException

Java Exception Handling Keywords

- **throw**

- Used to **explicitly create and throw an exception** in code.
- Transfers control to the runtime for handling.
- Commonly used for **custom validations** (e.g., null password, invalid input).
- **Example:**

```
if (password == null) {  
    throw new IllegalArgumentException("Password cannot be null");  
}
```

Java Exception Handling Keywords

- **try-catch**

- Used to **handle exceptions**.
 - try → contains risky code.
 - catch → handles the exception.
 - Multiple catch blocks allowed.
 - Can be nested.
 - catch parameter must be an Exception type.
- **Example:**

```
try {  
    int x = 10 / 0;  
} catch (ArithmeticException e) {  
    System.out.println("Cannot divide by zero");  
}
```

Java Exception Handling Keywords

- **try-with multiple catch blocks:**

- **Multiple catch blocks** allow handling **different types of exceptions separately**.
- Java checks the catch blocks **in order** and executes the first one that matches the exception type.
- This is useful when a method can throw **more than one kind of exception**.

- **Example:**

```
try {  
    // risky code that may throw exceptions  
} catch (ExceptionType1 e) {  
    // handle ExceptionType1  
} catch (ExceptionType2 e) {  
    // handle ExceptionType2  
} catch (Exception e) {  
    // handle any other exception (optional)  
}
```

Java Exception Handling Keywords

- try-with multiple catch blocks:

Error: Cannot divide by zero
Program continues normally

```
import java.io.*;
public class MultipleCatchExample {

    public static void main(String[] args) {
        try {
            // May throw ArithmeticException
            int result = 10 / 0;

            // May throw ArrayIndexOutOfBoundsException
            int[] numbers = {1, 2, 3};
            System.out.println(numbers[5]);

        } catch (ArithmeticException e) {
            System.out.println("Error: Cannot divide by zero");
        } catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("Error: Array index out of bounds");
        } catch (Exception e) {
            System.out.println("Error: Some other exception
occurred");
        }

        System.out.println("Program continues normally");
    }
}
```


Java Exception Handling Keywords

- **throws**

- Declares exceptions that a method **may pass to its caller**.
- Used in the **method signature**.
- Allows the caller to either handle or further propagate the exception.
- Can be used with the main() method.

- **Example:**

```
public void readFile() throws IOException
```

- Can declare **multiple exceptions**.

```
returnType methodName() throws ExceptionType1, ExceptionType2 {  
    // risky code  
}
```

Examples

The Throw of multiple exceptions



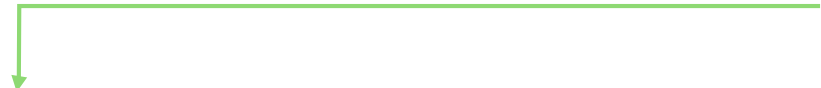
```
import java.io.*;

public class Test {

    static void readData() throws IOException, ClassNotFoundException {
        FileInputStream file = new FileInputStream("data.txt"); // May throw
        Class.forName("com.mysql.jdbc.Driver"); // May throw ClassNotFoundException
    }

    public static void main(String[] args) {
        try {
            readData();
        } catch (IOException | ClassNotFoundException e) {
            System.out.println("Exception handled: " + e);
        }
    }
}
```

The catch can catch multiple exceptions in a single catch block.



Java Exception Handling Keywords

- **finally**

- Optional block used with try-catch.
- Executes **always**, whether an exception occurs or not.
- Commonly used for **resource cleanup** (closing files, database connections).

- **Example:**

```
finally {  
    System.out.println("Cleanup code runs here");  
}
```

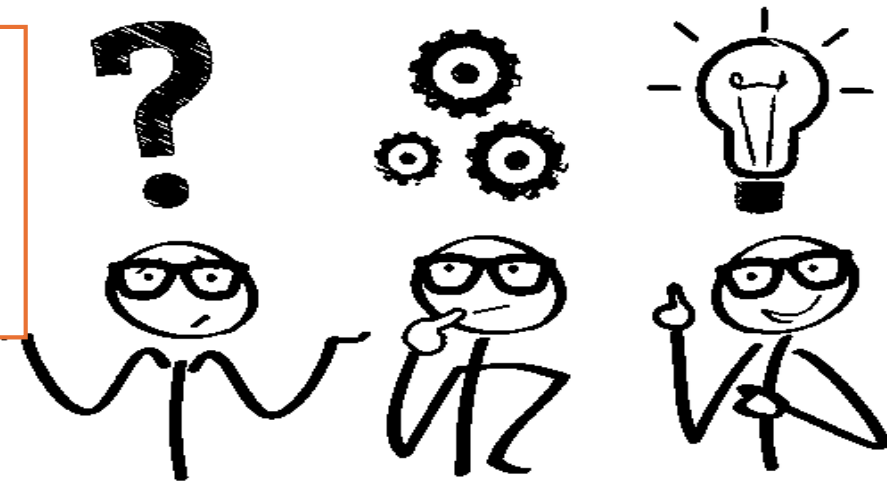
Useful Methods of Throwable

Method	Purpose
<code>getMessage()</code>	Returns the exception message provided when creating the exception.
<code>getLocalizedMessage()</code>	Returns a locale-specific message; by default, calls <code>getMessage()</code> .
<code>getCause()</code>	Returns the underlying cause of the exception, or null if unknown.
<code>toString()</code>	Returns a string containing the exception class name and the message.
<code>printStackTrace()</code>	Prints the stack trace to the standard error stream. Can also print to a <code>PrintStream</code> or <code>PrintWriter</code> for logging.

Problem Solving Steps

```
try {  
    int result = 10 / 0;  
} catch (ArithmeticException e) {  
    System.out.println("Message: " + e.getMessage());  
    System.out.println("Cause: " + e.getCause());  
    System.out.println("ToString: " + e.toString());  
    e.printStackTrace();  
}
```

```
Message: / by zero  
Cause: null  
ToString: java.lang.ArithmeticException: / by zero  
java.lang.ArithmeticException: / by zero  
    at Main.main(Main.java:3)
```



A Custom Exception Class

- Java provides many built-in exception classes, but sometimes **custom exceptions** are needed.
- **Purpose:** Notify the caller about a **specific type of error** with a meaningful message or additional information.
- **Custom fields** (like error codes) can be added to provide more context about the error.
- Example scenario: A method processes **only text files**; if another file type is passed, a **custom exception** can provide an **appropriate message and error code**.



A Custom Exception Class

Error: Only text files are allowed | Code: 101

```
class MyException extends Exception {
    private int errorCode;

    public MyException(String message, int errorCode) {
        super(message);
        this.errorCode = errorCode;
    }

    public int getErrorCode() {
        return errorCode;
    }
}

public class FileProcessor {
    public static void processFile(String fileName) throws MyException {
        if (!fileName.endsWith(".txt")) {
            throw new MyException("Only text files are allowed", 101);
        }
        System.out.println("Processing file: " + fileName);
    }

    public static void main(String[] args) {
        try {
            processFile("data.pdf");
        } catch (MyException e) {
            System.out.println("Error: " + e.getMessage() + " | Code: " +
e.getErrorCode());
        }
    }
}
```


Assignment

- Write a Java program that performs operations on an integer array. The program should handle multiple types of exceptions gracefully.
- **Requirements**
 - Ask the user to input the **size of the array** and its **elements**.
 - Then ask the user for an **index** to retrieve an element.
 - If the user enters an **invalid index**, handle **ArrayIndexOutOfBoundsException**.
 - Ask the user to input **two integers** to divide.
 - Handle **ArithmeticException** if the second number is zero.
 - Create a custom exception **NegativeArraySizeException** if the user enters a negative array size, and display a **meaningful message and error code**.
 - Use **multiple catch blocks**.
 - Include a **finally block** to print "Program completed" regardless of exceptions.
 - Use methods for array input and division, and **propagate exceptions using throws** where appropriate.



Thank you

**BE
THE
EXCEPTION**